

SET2 A1. Временная сложность рекурсии

1. Составление рекурентного соотношения

algorithm1(A, n)

if $n \leq 20 \rightarrow O(1)$ - проверка условия

return A[n] $\rightarrow O(1)$ - доступ к массиву

$x = \text{algorithm1}(A, n - 5) \rightarrow$ Рекурсивный вызов: $T(n - 5)$

for $i = 1$ to $\lfloor n/2 \rfloor \rightarrow$ Внешний цикл: $n/2$ итераций

for $j = 1$ to $\lfloor n/2 \rfloor \rightarrow$ Внутренний цикл: $n/2$ итераций для каждого i

$A[i] = A[i] - A[j] \rightarrow O(1)$ - арифметическая операция

$x = x + \text{algorithm1}(A, n - 8) \rightarrow$ Рекурсивный вызов: $T(n - 8)$

return x $\rightarrow O(1)$

ВЫВОД:

Базовый случай: $O(1)$ для $n \leq 20$

Первый рекурсивный вызов: $T(n - 5)$

Двойной цикл:

Внешний цикл: $\lfloor n/2 \rfloor = \Theta(n)$ итераций

Внутренний цикл: $\lfloor n/2 \rfloor = \Theta(n)$ итераций для каждого i

Каждая итерация: $O(1)$ операций

Общая сложность циклов: $\Theta(n) \times \Theta(n) \times O(1) = \Theta(n^2)$

Второй рекурсивный вызов: $T(n - 8)$

Возврат значения: $O(1)$

$$T(n) = T(n - 5) + T(n - 8) + O(n^2) + O(1) = T(n - 5) + T(n - 8) + \Theta(n^2), T(n) = O(1) \text{ для } n \leq 20$$

*Дополнительное пояснение:

Нерекурсивная часть имеет сложность $\Theta(n^2)$, потому что количество итераций цикла ограничено снизу функцией $c_1 \times n^2$ и сверху функцией $c_2 \times n^2$ для некоторых $\text{const } c_1$ и $c_2 > 0$ n и больших n . Остальные операции добавляют только $O(1)$, что в свою очередь перекрываются $\Theta(n^2)$

algorithm2(A, n)

if $n \leq 50 \rightarrow O(1)$ - проверка условия

return A[n] $\rightarrow O(1)$ - доступ к массиву

$x = \text{algorithm2}(A, \lfloor n/4 \rfloor) \rightarrow$ Рекурсивный вызов: $T(\lfloor n/4 \rfloor)$

for $i = 1$ to $\lfloor n/3 \rfloor \rightarrow O(\lfloor n/3 \rfloor) \rightarrow$ Цикл: $n/3$ итерации

$A[i] = A[n - i] - A[i] \rightarrow O(1) \rightarrow$ арифметика и доступ к элементу

$x = x + \text{algorithm2}(A, \lfloor n/4 \rfloor) \rightarrow$ Рекурсивный вызов: $T(\lfloor n/4 \rfloor)$

return x $\rightarrow O(1)$

ВЫВОД:

Базовый случай: $O(1)$ для $n \leq 50$

Первый рекурсивный вызов: $T(\lceil n/4 \rceil)$

Цикл:

Количество итераций: $\lceil n/3 \rceil = \Theta(n)$

Каждая итерация: $O(1)$ операций

Общая сложность цикла: $\Theta(n) \times O(1) = \Theta(n)$

Второй рекурсивный вызов: $T(\lceil n/4 \rceil)$

Возврат значения: $O(1)$

Суммируем:

$$T(n) = 2T(\lceil n/4 \rceil) + \Theta(n), T(n) = O(1) \text{ для } n \leq 50$$

*Дополнительное пояснение аналогично пояснению для algorithm1

2. Асимптотическая оценка

Используем метод дерева рекурсии для оценки algorithm1

Уровень 0: $T(n)$ [1 вершина]

Уровень 1: $T(n-5)$ $T(n-8)$ [2 вершины]

Уровень 2: $T(n-10)$ $T(n-13)$ $T(n-13)$ $T(n-16)$ [4 вершины]

Анализ дерева:

Максимальное количество вершин на уровне $m = 2^m$. Максимальный шаг = 5, значит глубина приблизительно $n/5$. Размер задачи на уровне m в худшем случае $n - 5m$, в лучшем $n - 8m$. Так как количество вершин $\leq 2^m$ и размер задачи приблизительно $n - 5m$, работа на уровне $m \leq$

$$2^m \times \Theta((n - 5m)^2), \text{ тогда общая работа: } T(n) \leq \sum_{m=1}^{n/5} 2^m \times \Theta((n - 5m)^2)$$

Решение имеет вид $T(n) = \Theta(r^n)$, r - корень характеристического уравнения

$$r^n = r^{n-5} + r^{n-8}$$

$$r^8 = r^3 + 1$$

$$r \approx 1.123$$

$$T(n) = \Theta(1.123^n)$$

Используем мастер-теорему для оценки algorithm2

$$T(n) = aT(n/b) + f(n), a = 2, b = 4, f(n) = \Theta(n)$$

$$n^{\log_b a} = n^{\log_4 2} = n^{1/2}$$

Сравниваем $f(n) = \Theta(n)$ с $n^{1/2}$

(Случай 3) $f(n)$ растет быстрее, чем $n^{\log_b a}$, и выполняется условие регулярности $a \times f(n/b) \leq c \times f(n)$ для $c < 1$: $2 \times (n/4) = n/2 \leq c \times n$ для $c = 0.9$ - выполняется. Следовательно, $T(n) = \Theta(f(n)) = \Theta(n)$ - линейная.